

# 计算机视觉课程课后作业四

建智 2201 谢天赐 20221082044

## 问题一 4.1:

实现代码如图所示:

```
window = tk.Tk()
window.title("计算机视觉作业任务")

# 设置窗体大小
window.geometry("400x200")

# 定义按钮功能
def execute_task():
    input_value = entry.get()
    if input_value:
        messagebox.showinfo("执行结果", f"输入的参数是: {input_value}")
    else:
        messagebox.showwarning("警告", "请输入参数!")

# 创建标签
label = tk.Label(window, text="请输入参数:")
label.pack(pady=10)

# 创建文本框
entry = tk.Entry(window, width=30)
entry.pack(pady=10)

# 创建执行按钮
button = tk.Button(window, text="执行任务", command=execute_task)
button.pack(pady=20)
```

图 1

运行效果如图 2 所示:

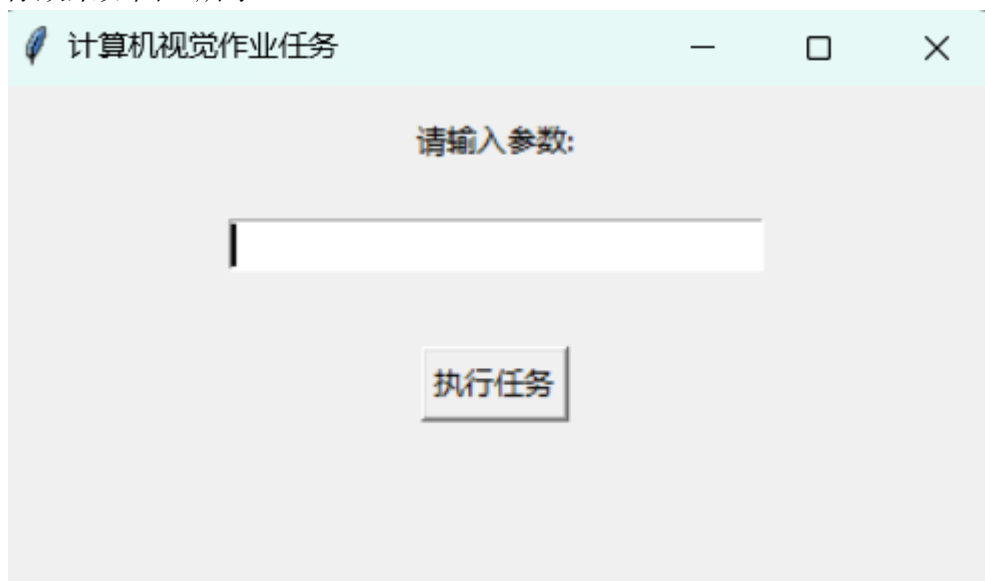


图 2

点击执行任务按钮效果如图 3 所示。

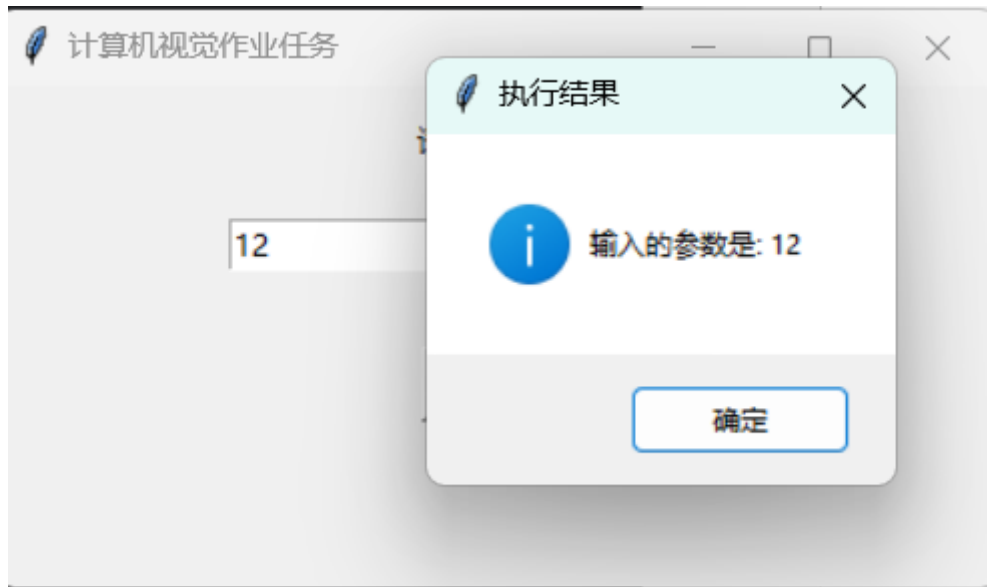


图 3

## 问题二 4.2:

在进行图像处理时，使用 OpenCV 库的 `cv2.resize()` 函数调整图像大小时，可能会遇到错误信息：“(-215:Assertion failed) !ssize.empty()”。这个错误指出在执行 `resize()` 操作时，输入图像为空。这种情况通常发生在图像变量 `img` 在调用 `cv2.resize()` 之前没有被正确加载。

以下是导致这种情况的几种可能原因及其解决方案：

### 1) 图像加载失败 (`img = None`):

当使用 `cv2.imread()` 函数读取图像文件时，如果提供的文件路径不正确，或者图像文件不存在，OpenCV 将返回 `None`。这意味着图像没有被成功加载，但程序仍尝试对一个空对象执行 `resize()` 操作，从而引发错误。

解决方案：确保文件路径正确，文件存在且可访问。

文件路径错误或文件不存在：

### 2) 如果指定的图像路径不正确，或者文件已被移动或删除，程序将无法找到并加载图像。

解决方案：使用绝对路径来指定图像文件，以避免相对路径可能引起的问题。

### 3) 文件权限或格式问题：

如果图像文件存在访问权限问题，程序可能无法读取图像。

解决方案：检查文件权限，确保程序有足够的权限读取文件。

### 4) 如果文件格式不被支持或文件已损坏，OpenCV 同样无法加载图像。

解决方案：确保文件格式受支持且文件未损坏。

### 5) 路径中的特殊字符：

如果路径包含特殊字符或非 ASCII 字符（如中文），可能会导致路径解析错误，从而无法加载图像。

解决方案：确保路径中不包含特殊或非 ASCII 字符，或使用适当的编码处理路径字符串。

代码解决方案：在尝试调整图像大小时，首先检查图像是否已成功加载：

```

if img is None:
    # 使用此方法重新读取图像
    # 参数 -1 表示按原图读取, 1 表示读取 BGR, 0 为灰度图
    img=cv2.imdecode(np.fromfile(fnImg, dtype=np.uint8),
cv2.IMREAD_COLOR)

```

这段代码首先检查 `img` 是否为 `None`, 如果是, 则使用 `np.fromfile()` 读取图像文件为字节流, 然后使用 `cv2.imdecode()` 将其解码为图像。

### 问题三 4.3:

在图像处理中, 选择合适的阈值对于图像分割和特征提取至关重要。在本例中, ID:6 (阈值 127) 的图像显示效果相对较好, 车牌号的字母和数字部分非常清晰, 背景噪声相对较少。这表明 127 是一个合适的阈值。仅从例图来看:



图 4 示例

为了找到更精确的阈值范围, 可以改变步长, 以 127 为基准, 将步长改为 1, 测试阈值在 122 至 132 之间的效果。通过这种方法, 可以确定一个更精确的阈值范围, 以获得最佳的图像处理效果。

当然, 也可以使用 Kmeans 聚类算法或者是 SVM 支持向量机等深度学习、机器学习手段来选择更为精确的阈值范围, 然而工作量较大, 本作业不做展示。

### 问题四 4.4:

在处理用户输入或文件读取的数据时, 可能会遇到错误信息, 提示无法将字符串转换为整数, 因为字符串包含不合法的字符。这通常发生在使用 `int()` 函数尝试将包含换行符、空格或其他非数字字符的字符串转换为整数时。

**解决方案:** 在尝试将字符串转换为整数之前, 先使用 `.strip()` 方法去除字符串中的换行符和空白符。这可以确保字符串只包含合法的数字字符, 从而避免转换错误。

```
cleaned_string = input_string.strip()
integer_value = int(cleaned_string)
```

这段代码首先使用 `.strip()` 方法清除字符串中的所有前后空白字符，然后安全地将清理后的字符串转换为整数。